

Rapport de Mini-Projet : Labyrinthe d'Ayutthaya

Cours : Algorithme Avancée (IF05XO40)

Réaliser par : **Mamadou NIMAGA**
Identifiant Étudiant : 22518134

Professeur : **Nicolas LOMENIE**

Date : 20/12/2025

Table des matières

<u>1. Introduction</u>	<u>2</u>
<u>2. Architecture et génie logiciel</u>	<u>2</u>
<u>3. Algorithmes implémentés</u>	<u>2</u>
<u>3.1 Propagation du feu (BFS multi-sources)</u>	<u>2</u>
<u>3.2 Chemin de survie (A*)</u>	<u>3</u>
<u>4. Résultats et validation</u>	<u>5</u>
<u>5. Génie Logiciel et documentation (Javadoc)</u>	<u>8</u>
<u>6. Perspectives : Vers l'Apprentissage par Renforcement</u>	<u>8</u>
<u>7. Conclusion</u>	<u>9</u>

1. Introduction

Ce module traite du problème de l'évasion d'un prisonnier dans une grille de dimension $N * M$. La difficulté réside dans la présence d'un incendie qui se propage à chaque minute.

L'objectif est de déterminer par le calcul si un chemin de survie existe vers la sortie.

2. Architecture et génie logiciel

Une attention particulière a été portée à la robustesse et à l'expérience utilisateur :

- ★ **Gestion des entrées** : Le programme accepte un chemin de fichier en argument de ligne de commande pour faciliter les tests automatisés.
- ★ **Sécurité des fichiers** : Une vérification d'existence (`file.exists()`) a été intégrée pour éviter les plantages si dossier `data/` est manquant.
- ★ **Interface Graphique (GUI)** : Utilisation d'une fenêtre unique dynamique qui se redimensionne et se réinitialise à chaque instance pour éviter l'encombrement visuel.
- ★ **Fermeture propre** : L'utilisation de `System.exit(0)` garantit que le terminal rend la main immédiatement après l'exécution du JAR, même si des fenêtres graphiques sont restées ouvertes.

3. Algorithmes implémentés

3.1 Propagation du feu (BFS multi-sources)

Avant de calculer le chemin du prisonnier, nous simulons l'incendie.

- ★ **Méthode** : Un parcours en largeur (BFS) est lancé simultanément depuis toutes les sources de feu 'F'.
- ★ **Complexité** : $O(V + E)$, garantissant un pré-calcul instantané même sur de grandes grilles.
- ★ **Résultat** : Un tableau de temps d'arrivée du feu (**fireTimes**) est généré pour chaque cellule de la grille

Code :

```
/**
 * Simule l'expansion du feu sur la grille.
 *
 * @param g Le graphe du labyrinthe.
 * @param nl Nombre de lignes.
 * @param nc Nombre de colonnes.
 * @return Un tableau de doubles associant chaque index
de sommet au temps
 *         d'arrivée du feu.
 */
```

```

    private static double[] computeFireTimes(Graph g, int nl,
int nc) {
    double[] fireTimes = new double[nl * nc];
    Arrays.fill(fireTimes, Double.POSITIVE_INFINITY);
    Queue<Integer> q = new LinkedList<>();
    for (Vertex v : g.vertexlist)
        if (v.info == 'F') {
            fireTimes[v.num] = 0;
            q.add(v.num);
        }
    int[][] dirs = { { -1, 0 }, { 1, 0 }, { 0, -1 }, {
0, 1 } };
    while (!q.isEmpty()) {
        int u = q.poll();
        for (int[] d : dirs) {
            int ni = (u / nc) + d[0], nj = (u % nc) +
d[1];
            if (ni >= 0 && ni < nl && nj >= 0 && nj <
nc) {
                int v = ni * nc + nj;
                if (g.vertexlist.get(v).info != '#'
&& fireTimes[v] == Double.POSITIVE_INFINITY) {
                    fireTimes[v] = fireTimes[u] + 1;
                    q.add(v);
                }
            }
        }
    }
    return fireTimes;
}

```

3.2 Chemin de survie (A*)

Nous utilisons l'algorithme A* pour trouver le chemin le plus court tout en respectant la contrainte de survie.

- ★ **Heuristique de Manhattan** : Puisque les déplacements sont limités à la 4-connexité (Nord, Sud, Est, Ouest), nous utilisons la distance de Manhattan :

$$h(n) = |x_n - x_{but}| + |y_n - y_{but}|$$

Cette heuristique est plus précise que la distance euclidienne pour ce type de grille.

- ★ **Contrainte de temps** : A chaque étape, le prisonnier ne peut se déplacer vers une case que si son temps d'arrivée est **strictement inférieur** au temps d'arrivée du feu pré-calculé.

Code :

```
/**
 * Recherche le plus court chemin de survie via
 l'algorithme A*. Utilise la
 * distance de Manhattan comme heuristique :  $h(n) =$ 
  $|x_n - x_{but}| + |y_n -$ 
 *  $y_{but}|$ 
 *
 * @param g Le graphe du labyrinthe.
 * @param s Index du sommet de départ.
 * @param e Index du sommet de sortie.
 * @param nc Nombre de colonnes (pour les coordonnées).
 * @param ft Tableau des temps d'arrivée du feu
 pré-calculés.
 * @return La liste des sommets constituant le chemin de
 survie.
 */
private static LinkedList<Integer> AStar(Graph g, int s,
int e, int nc, double[] ft) {
    g.vertexlist.get(s).timeFromSource = 0;
    PriorityQueue<Integer> pq = new
PriorityQueue<>(Comparator.comparingDouble(
        v -> g.vertexlist.get(v).timeFromSource +
Math.abs(v % nc - e % nc) + Math.abs(v / nc - e / nc)));
    pq.add(s);
    HashSet<Integer> visited = new HashSet<>();
    while (!pq.isEmpty()) {
        int u = pq.poll();
        if (u == e)
            break;
        if (visited.contains(u))
            continue;
        visited.add(u);
        board.update(u);
        try {
            Thread.sleep(30);
        } catch (Exception ex) {
        }
        for (WeightedGraph.Edge edge :
g.vertexlist.get(u).adjacencylist) {
            int v = edge.destination;
            double arr =
g.vertexlist.get(u).timeFromSource + 1;
```

```

        // Condition de survie : arriver avant le
feu
        if (arr < ft[v] && arr <
g.vertexlist.get(v).timeFromSource) {
            g.vertexlist.get(v).timeFromSource =
arr;
            g.vertexlist.get(v).prev =
g.vertexlist.get(u);
            pq.add(v);
        }
    }
    LinkedList<Integer> p = new LinkedList<>();
    Vertex curr = g.vertexlist.get(e);
    while (curr != null) {
        p.addFirst(curr.num);
        curr = curr.prev;
    }
    return p;
}

```

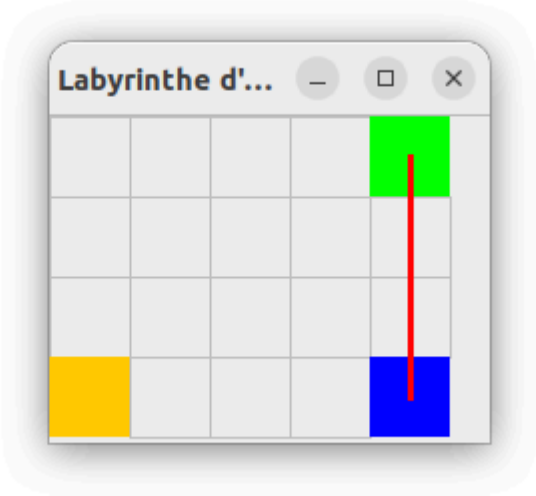
4. Résultats et validation

Le programme a été validé avec le fichier ayuttaya.txt dont le contenu est fourni dans le sujet.
 Les résultats obtenus en console sont conformes aux attentes :

★ Instance 1 : Y (Calcul terminé en 3 nœuds explorés).

```
App (1) [Java Application] /snap/eclipse/127/usr/lib/eclipse/plugins/org.eclipse.j
[INFO] Utilisation du fichier par défaut (dans le doss

--- Traitement de l'Instance 1 ---
    Number of nodes explored: 3
    Total time of the path: 3.0
Résultat : [Y]
```

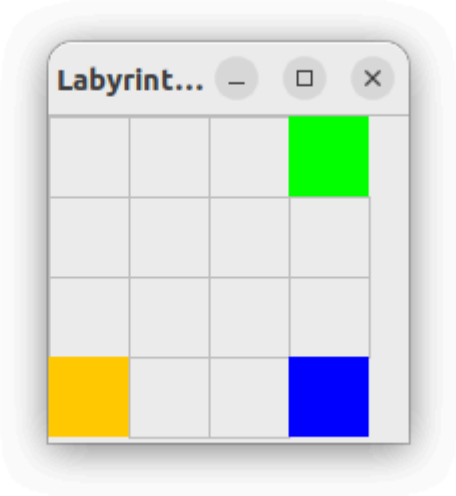


★ Instance 2 : N (Calcul terminé en 6 nœuds explorés - Le feu rattrape le prisonnier).

```
App (1) [Java Application] /snap/eclipse/127/usr/lib/eclipse/plugins/org.eclipse.j
[INFO] Utilisation du fichier par défaut (dans le doss

--- Traitement de l'Instance 1 ---
    Number of nodes explored: 3
    Total time of the path: 3.0
Résultat : [Y]

--- Traitement de l'Instance 2 ---
    Number of nodes explored: 6
    Total time of the path: Infinity
Résultat : [N]
```



★ Instance 3 : N (Calcul terminé en 1 nœud exploré - Prisonnier emmuré).

```
App (1) [Java Application] /snap/eclipse/127/usr/lib/eclipse/plugins/org.eclipse.j
[INFO] Utilisation du fichier par défaut (dans le doss

--- Traitement de l'Instance 1 ---
    Number of nodes explored: 3
    Total time of the path: 3.0
Résultat : [Y]

--- Traitement de l'Instance 2 ---
    Number of nodes explored: 6
    Total time of the path: Infinity
Résultat : [N]

--- Traitement de l'Instance 3 ---
    Number of nodes explored: 1
    Total time of the path: Infinity
Résultat : [N]
```



5. Génie Logiciel et documentation (Javadoc)

Suivant une approche de Génie Logiciel, le projet a été versionné avec **Git**. Une attention particulière a été portée à la documentation :

- ★ Utilisation des tags **Javadoc** (`@param`, `@return`) pour chaque méthode.
- ★ Génération d'un site de documentation technique dans le dossier **/doc**.

6. Perspectives : Vers l'Apprentissage par Renforcement

Bien que l'algorithme A* avec heuristique de Manhattan soit optimal pour ce problème déterministe, une évolution intéressante du projet, serait l'exploration de **l'Intelligence Artificielle** via le **Q-learning** (Apprentissage par renforcement).

- **Limitation actuelle** : Notre approche actuelle repose sur un graphe statique où les temps d'arrivée du feu sont pré-calculés par un BFS.
- **Apport du Q-learning** : L'utilisation d'un agent apprenant permettrait de résoudre le problème dans des environnements où la propagation du feu est stochastique

(aléatoire) ou inconnue à l'avance. L'agent apprendrait, par essais et erreurs, une **politique** optimale pour maximiser ses chances de survie en fonction des récompenses reçues lors de ses déplacements.

7. Conclusion

L'implémentation allie efficacité algorithmique et rigueur logicielle. Le choix de **l'algorithme A* avec heuristique de Manhattan** assure une recherche de chemin optimale, tandis que le **pré-calcul BFS** du feu garantit la sécurité du trajet proposé.